

COIT20258 Software Engineering

Week 4

**Architectural Design
& O-O design**

Architectural Design

Topics

- ✧ Architectural design decisions
- ✧ Architectural views
- ✧ Architectural patterns
- ✧ Application architectures
- ✧ Object-oriented design using the UML

Architectural Design

- ✧ Architectural design is concerned with understanding how a software system should be organized & designing the *overall structure* of that system
- ✧ Architectural design is the *critical link between design & requirements engineering*, as it identifies the main *structural components* in a system & the relationships between them
- ✧ The output of the architectural design process is an *architectural model* that describes how the system is organized as a set of communicating components

Architectural Design Decisions

✧ Performance

- Localize critical operations & minimize communications. Use large rather than fine-grain components.

✧ Security

- Use a layered architecture with critical assets in the inner layers

✧ Safety

- Localize safety-critical features in a small number of sub-systems

✧ Availability

- Include redundant components & mechanisms for fault tolerance

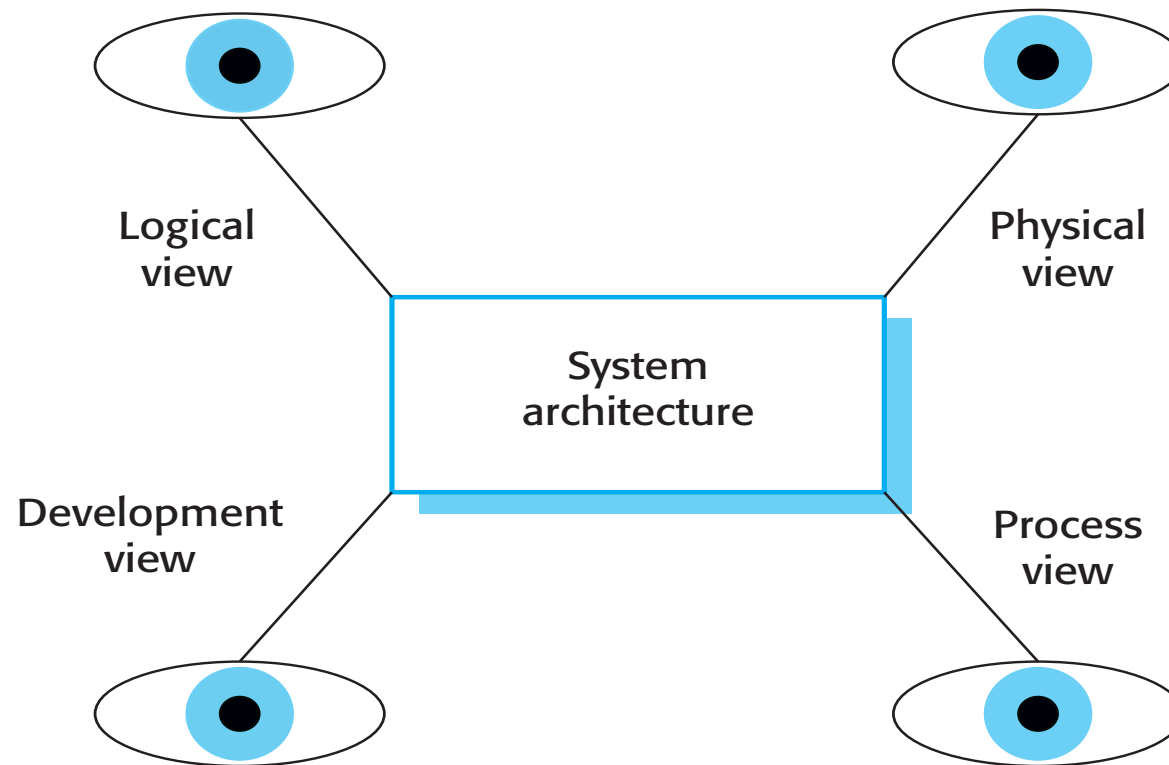
✧ Maintainability

- Use fine-grain, replaceable components

4+1 View Model of Software Architecture

- ✧ A **logical view**, which shows the key abstractions in the system as objects or object classes
- ✧ A **process view**, which shows how, *at run-time*, the system is composed of interacting processes
- ✧ A **development view**, which shows how the software is decomposed for development
- ✧ A **physical view**, which shows the system hardware & how software components are distributed across the processors in the system
- ✧ Related using **use cases** or **scenarios (+1)**

Architectural Views



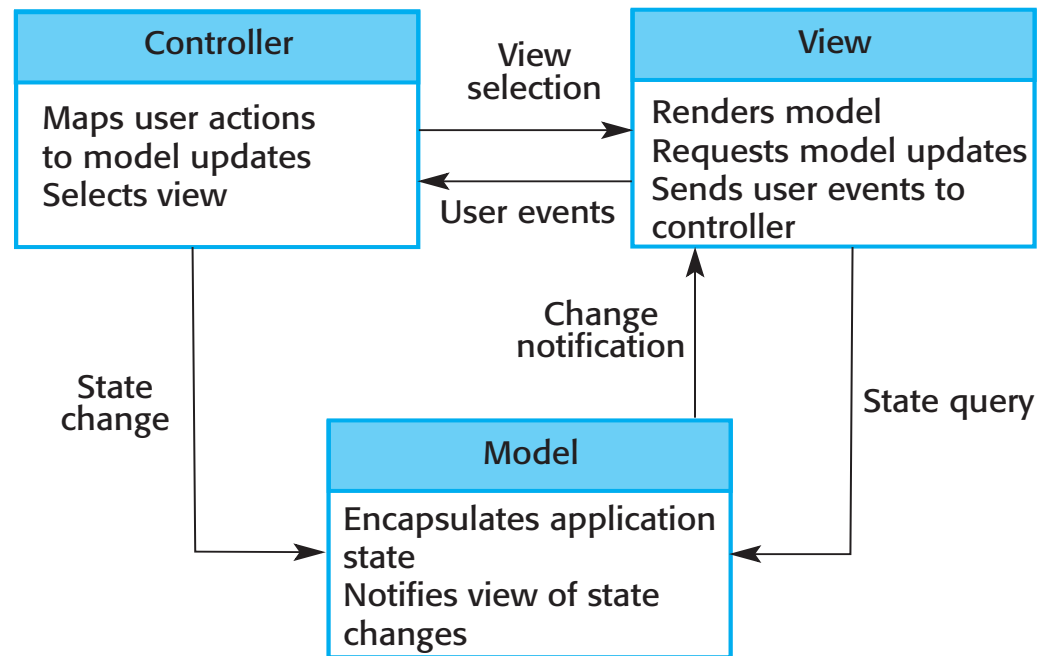
Architectural Patterns

- ✧ *Patterns* are a means of representing, sharing & reusing knowledge
- ✧ An architectural pattern is a stylized description of *good design practice*, which has been tried & tested in different environments
- ✧ Patterns should include information about when they are & when they are not useful
- ✧ Patterns may be represented using tabular & graphical descriptions

Model-View-Controller (MVC) Pattern

Name	MVC (Model-View-Controller)
Description	Separates presentation & interaction from the system data. The system is structured into three logical components that interact with each other. The Model component manages the system data & associated operations on that data. The View component defines & manages how the data is presented to the user. The Controller component manages user interaction (e.g., key presses & mouse clicks) & passes these interactions to the View & the Model.
Example	Figure 6.6 shows the architecture of a web-based application system organized using the MVC pattern.
When used	Used when there are <i>multiple ways to view & interact with data</i> . Also used when the future requirements for interaction & presentation of data are unknown.
Advantages	Allows the data to change <i>independently</i> of its representation & vice versa. Supports presentation of the same data in different ways with changes made in one representation shown in all of them.
Disadvantages	Can involve additional code & code complexity when the data model & interactions are simple.

Organization of the Model-View-Controller



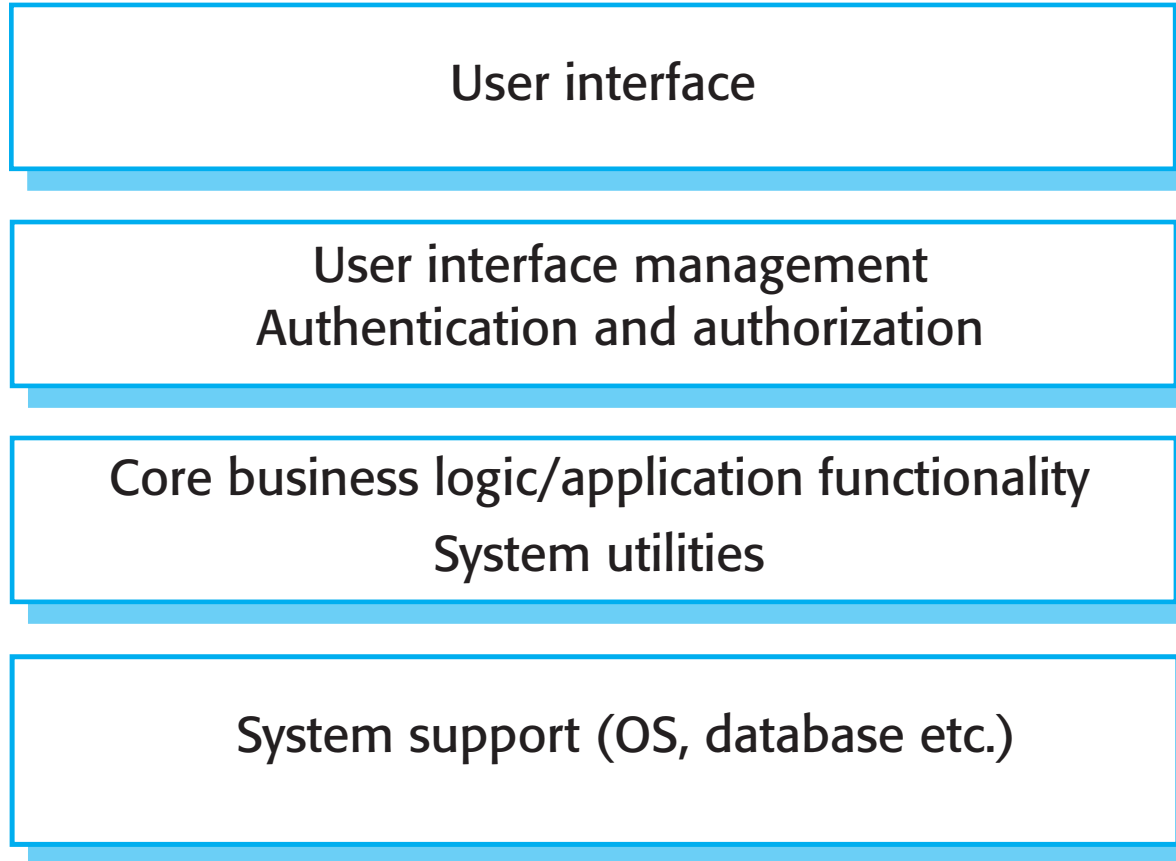
Layered Architecture

- ✧ Used to model the *interfacing of subsystems*
- ✧ Organizes the system into a set of layers (or abstract machines); each of which provides a set of services
- ✧ Supports the *incremental development* of subsystems in different layers. When a layer interface changes, only the adjacent layer is affected.

Layered Architecture Pattern

Name	Layered architecture
Description	Organizes the system into layers with related functionality associated with each layer. A layer provides services to the layer above it, so the lowest-level layers represent core services that are likely to be used throughout the system.
Example	A layered model of a system for sharing copyright documents held in different libraries, as shown in Figure 6.9.
When used	Used when building new facilities on top of existing systems; when the development is spread across several teams with each team responsibility for a layer of functionality; when there is a requirement for multi-level security.
Advantages	Allows replacement of entire layers so long as the interface is maintained. Redundant facilities (e.g., authentication) can be provided in each layer to increase the dependability of the system.
Disadvantages	In practice, providing a clean separation between layers is often difficult & a high-level layer may have to interact directly with lower-level layers rather than through the layer immediately below it. Performance can be a problem because of multiple levels of interpretation of a service request as it is processed at each layer.

A “Generic” Layered Architecture



Repository Architecture

- ✧ Subsystems must *exchange data*. This may be done in two ways:
 - Shared data is held in a central database or repository & may be accessed by all sub-systems
 - Each subsystem maintains its own database & passes data explicitly to other subsystems
- ✧ When *large amounts of data are to be shared*, the repository model of sharing is most commonly used as this is an efficient data sharing mechanism

Repository Pattern

Name	Repository
Description	All data in a system is managed in a central repository that is accessible to all system components. Components do not interact directly, only through the repository.
Example	Figure 6.11 is an example of an IDE where the components use a repository of system design information. Each software tool generates information which is then available for use by other tools.
When used	You should use this pattern when you have a system in which large volumes of information are generated that has to be stored for a long time. You may also use it in data-driven systems where the inclusion of data in the repository triggers an action or tool.
Advantages	Components can be independent—they do not need to know of the existence of other components. Changes made by one component can be propagated to all components. All data can be managed consistently (e.g., backups done at the same time) as it is all in one place.
Disadvantages	The repository is a single point of failure so problems in the repository affect the whole system. May be inefficiencies in organizing all communication through the repository. Distributing the repository across several computers may be difficult.

Client-Server Architecture

- ✧ *Distributed system model* which shows how data & processing is distributed across a range of components
 - Can be implemented on a single computer
- ✧ Set of standalone servers which provide specific services such as printing & data management
- ✧ Set of clients which call on these services
- ✧ Network which allows clients to access servers

Client-Server Pattern

Name	Client-server
Description	In a client-server architecture, the functionality of the system is organized into services, with each service delivered from a separate server. Clients are users of these services & access servers to make use of them.
Example	Figure 6.11 is an example of a film & video/DVD library organized as a client–server system.
When used	Used when data in a shared database has to be accessed from a range of locations. Because servers can be replicated, may also be used when the load on a system is variable.
Advantages	The principal advantage of this model is that servers can be distributed across a network. General functionality (e.g., a printing service) can be available to all clients & does not need to be implemented by all services.
Disadvantages	Each service is a single point of failure so susceptible to denial of service attacks or server failure. Performance may be unpredictable because it depends on the network as well as the system. May be management problems if servers are owned by different organizations.

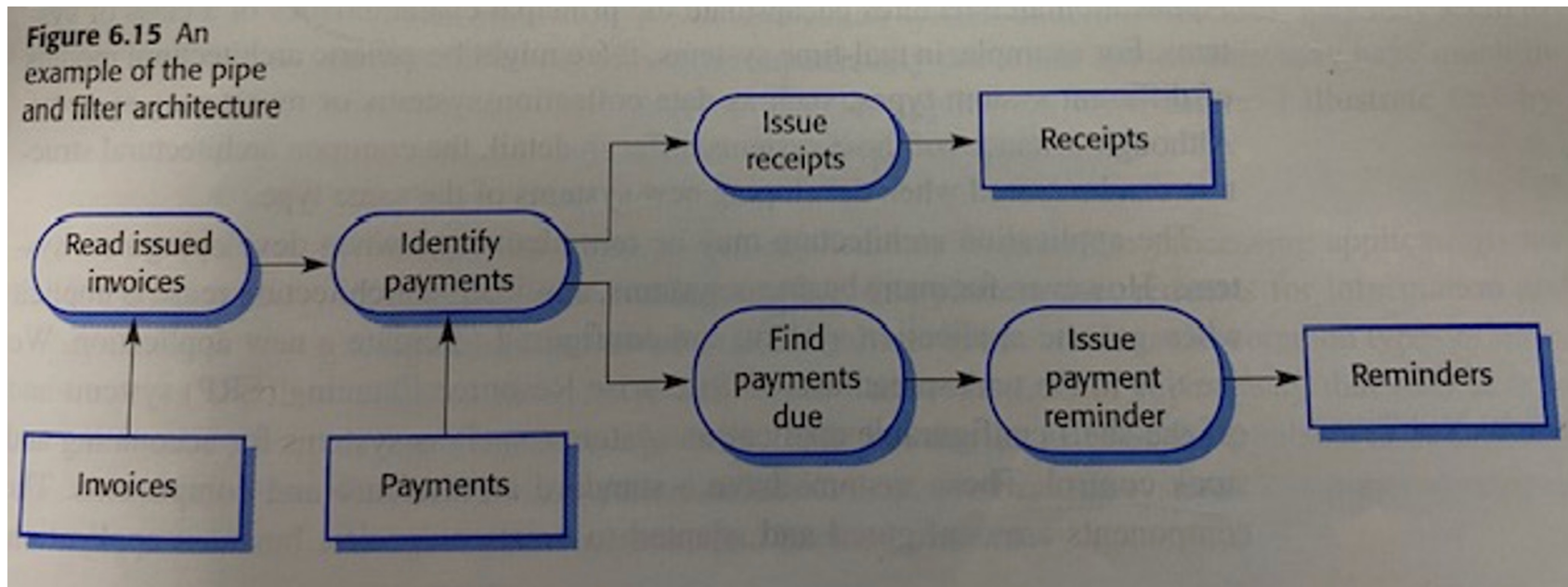
Pipe & Filter Architecture

- ✧ *Functional transformations* process their inputs to produce outputs
- ✧ May be referred to as a *pipe & filter model* (as in UNIX shell)
- ✧ Variants of this approach are very common. When transformations are sequential, this is a batch sequential model which is extensively used in data processing systems.
- ✧ Not really suitable for interactive systems

Pipe & Filter Pattern

Name	Pipe & filter
Description	The processing of the data in a system is organized so that each processing component (filter) is discrete & carries out one type of data transformation. The data flows (as in a pipe) from one component to another for processing.
Example	Figure 6.15 is an example of a pipe & filter system used for processing invoices.
When used	Commonly used in data processing applications (both batch- & transaction-based) where inputs are processed in separate stages to generate related outputs.
Advantages	Easy to understand & supports transformation reuse. Workflow style matches the structure of many business processes. Evolution by adding transformations is straightforward. Can be implemented as either a sequential or concurrent system.
Disadvantages	The format for data transfer has to be agreed upon between communicating transformations. Each transformation must parse its input & unparse its output to the agreed form. This increases system overhead & may mean that it is impossible to reuse functional transformations that use incompatible data structures.

An example of the Pipe & Filter Architecture Used in a Payments System



Application Architectures

- ✧ Application systems are designed to meet an *organizational need*
- ✧ As businesses have much in common, their application systems also tend to have a common architecture that reflects the application requirements
- ✧ A generic application architecture is an architecture for a type of software system that may be configured & adapted to create a system that meets specific requirements

Use of Application Architectures

- ✧ As a *starting point* for architectural design
- ✧ As a design checklist
- ✧ As a way of organizing the work of the development team
- ✧ As a means of assessing components for reuse
- ✧ As a vocabulary for talking about application types

Examples of Application Types

✧ Data processing applications

- *Data-driven* applications that process data in batches without explicit user intervention during the processing

✧ Transaction processing applications

- *Data-centred* applications that process user requests & update information in a system database

✧ Event processing systems

- Applications where system actions depend on interpreting events from the system's environment

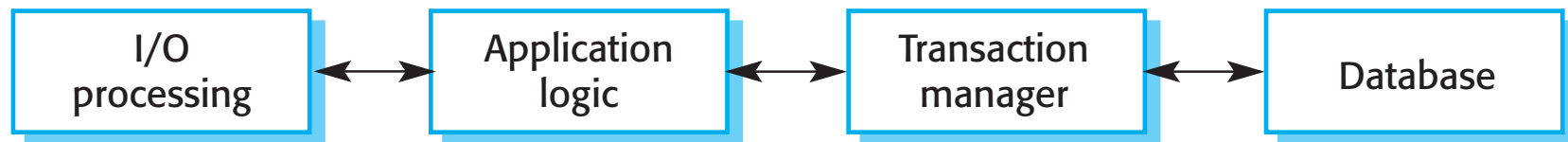
✧ Language processing systems

- Applications where the users' intentions are specified in a formal language that is processed & interpreted by the system

Transaction Processing Systems

- ✧ Process user requests for information from a database or requests to update the database
- ✧ From a user perspective a transaction is:
 - Any coherent sequence of operations that satisfies a goal;
 - For example - find the times of flights from London to Paris
- ✧ Users make asynchronous requests for service which are then processed by a transaction manager

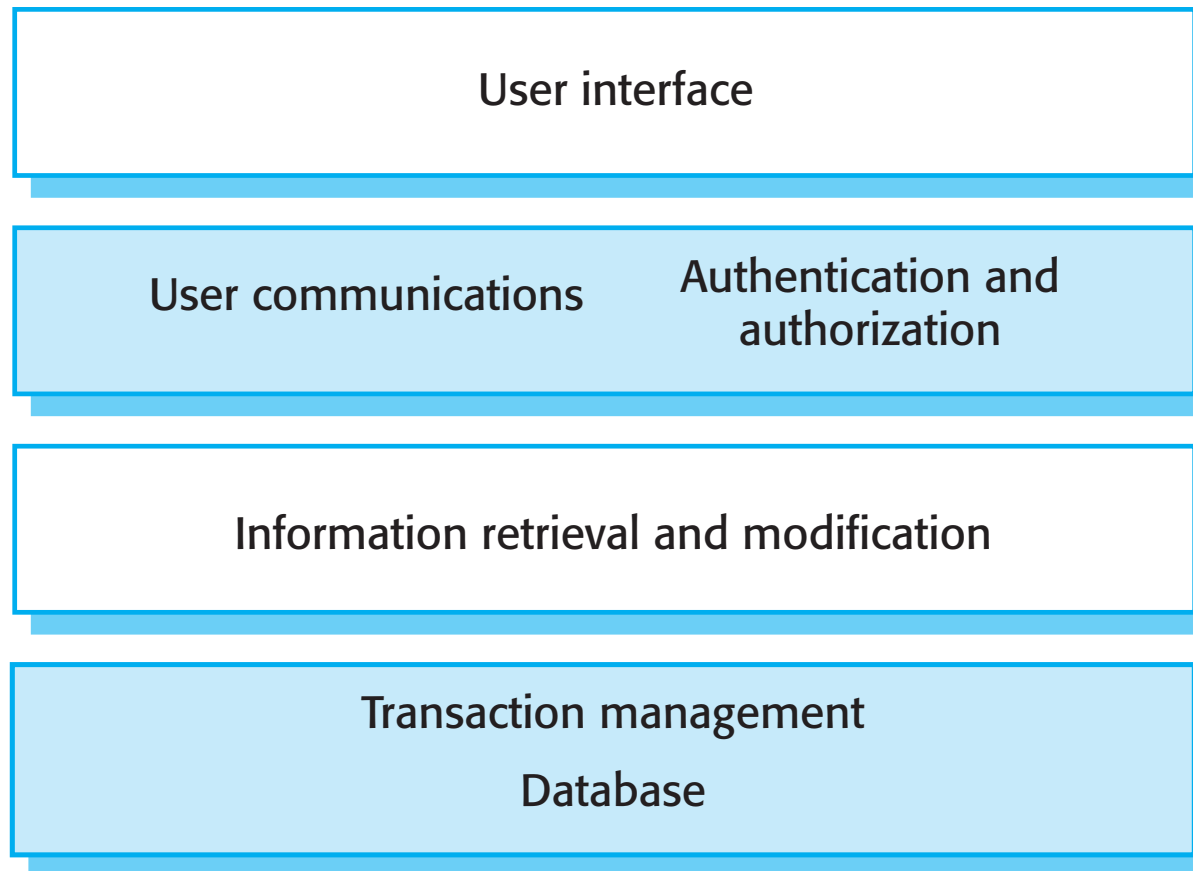
Structure of Transaction Processing Applications



Information Systems

- ✧ Information systems have a generic architecture that can be organized as a layered architecture
- ✧ These are transaction-based systems as interaction with these systems generally involves *database transactions*
- ✧ Layers include:
 - The user interface
 - User communications
 - Information retrieval
 - System database

Layered Information System Architecture



Web-Based Information Systems

- ✧ Information & resource management systems are now usually web-based systems where the user interfaces are implemented using a *web browser*
- ✧ For example, e-commerce systems are Internet-based resource management systems that accept electronic orders for goods or services & then arrange delivery of these goods or services to the customer
- ✧ In an e-commerce system, the application-specific layer includes additional functionality supporting a “shopping cart” in which users can place a number of items in separate transactions, then pay for them all together in a single transaction

Server Implementation

- ✧ These systems are often implemented as multi-tier client server/architectures
- ✧ The web server is responsible for all user communications, with the user interface implemented using a web browser
 - The *application server* is responsible for implementing application-specific logic as well as information storage & retrieval requests
 - The *database server* moves information to & from the database & handles transaction management

Language Processing Systems

- ✧ Accept a natural or artificial language as input & generate some other representation of that language
- ✧ May include an *interpreter* to act on the instructions in the language that is being processed

Object-Oriented (O-O) Design

An O-O Design Process

- ✧ Structured object-oriented design processes involve developing a number of different system models
- ✧ They require a lot of effort for development & maintenance of these models &, for small systems, this may not be cost-effective
- ✧ However, for large systems developed by different groups, design models are an important communication mechanism

Process Stages

- ✧ There are a variety of different object-oriented design processes that depend on the organization using the process
- ✧ Common activities in these processes include:
 - Define the *context & modes* of use of the system
 - Design the *system architecture*
 - Identify the *principal system objects*
 - Develop *design models*
 - Specify *object interfaces*

System Context & Interactions

- ✧ Understanding the relationships between the software that is being designed & its external environment is essential for deciding how to provide the required system functionality & how to structure the system to communicate with its environment
- ✧ Understanding of the context also lets you establish the *boundaries* of the system. Setting the system boundaries helps you decide what features are implemented in the system being designed & what features are in other associated systems.

Architectural Design

- ✧ Once interactions between the system & its environment have been understood, you use this information for designing the system architecture
- ✧ You identify the major components that make up the system & their interactions, & then may organize the components using an *architectural pattern* such as a layered or client-server model

Object Class Identification

- ✧ Identifying object classes is often a difficult part of O-O design
- ✧ There is no “magic formula” for object identification. It relies on the skill, experience & domain knowledge of system designers.
- ✧ Object identification is an *iterative* process. You are unlikely to get it right first time.

Identification Approaches

- ✧ Use a grammatical approach based on a *natural language description* of the system
- ✧ Based on the identification on *tangible things* in the application domain
- ✧ Use a behavioural approach & identify objects based on what participates in what *behaviour*
- ✧ Use a *scenario-based analysis*. The objects, attributes & methods in each scenario are identified.

Design Models

- ✧ Design models show the objects & object classes & relationships between these entities
- ✧ There are two kinds of design model:
 - *Structural models* describe the static structure of the system in terms of object classes & relationships
 - *Dynamic models* describe the dynamic interactions between objects

Examples of Design Models

- ✧ Subsystem models that show logical groupings of objects into coherent subsystems
- ✧ Sequence models that show the sequence of object interactions
- ✧ State machine models that show how individual objects change their state in response to events
- ✧ Other models include, e.g., use-case models, aggregation models, & generalization models

Subsystem Models

- ✧ Shows how the design is organized into logically related groups of objects
- ✧ In the UML, these are shown using packages - an encapsulation construct. This is a logical model. The actual organisation of objects in the system may be different.

Sequence Models

- ✧ Sequence models show the sequence of object interactions that take place
 - Objects are arranged horizontally across the top
 - Time is represented vertically so models are read top to bottom
 - Interactions are represented by labelled arrows, Different styles of arrow represent different types of interaction.
 - A thin rectangle in an object lifeline represents the time when the object is the controlling object in the system

An Illustrative Example

Examining the Requirements Document

- A requirements document specifies the purpose of the ATM system & what it must do
- Requirements Document
 - A local bank intends to install a new automated teller machine (ATM) to allow users (i.e., bank customers) to perform basic financial transactions
 - Each user can have only one account at the bank
 - ATM users
 - view their account balance
 - withdraw cash
 - deposit funds

... Examining the Requirements Document

- ATM user interface:
 - a screen that displays messages to the user
 - a keypad that receives numeric input from the user
 - a cash dispenser that dispenses cash to the user
 - a deposit slot that receives deposit envelopes from the user
- The cash dispenser begins each day loaded with 500 \$20 bills

... Examining the Requirements Document

- Upon first approaching the ATM, the user should experience the following sequence of events:
 - The screen displays “Welcome!” & prompts the user to enter an account number
 - The user enters a five-digit account number using the keypad
 - The screen prompts the user to enter the PIN (personal identification number) associated with the specified account number
 - The user enters a five-digit PIN using the keypad
 - If the user enters a valid account number & the correct PIN for that account, the screen displays the main menu. If the user enters an invalid account number or an incorrect PIN, the screen displays an appropriate message, then the ATM returns to *Step 1* to restart the authentication process.

... Examining the Requirements Document

- The following steps describe what occurs when the user enters 2 to make a withdrawal:
 - The screen displays a menu of standard withdrawal amounts & an option to cancel the transaction
 - The user enters a menu selection using the keypad
 - If the withdrawal amount is greater than the user's account balance, the screen displays a message stating this & telling the user to choose a smaller amount. The ATM then returns to Step 1. If the withdrawal amount chosen is less than or equal to the user's account balance (i.e., an acceptable amount), the ATM proceeds to Step 4. If the user chooses to cancel, the ATM displays the main menu & waits for user input.

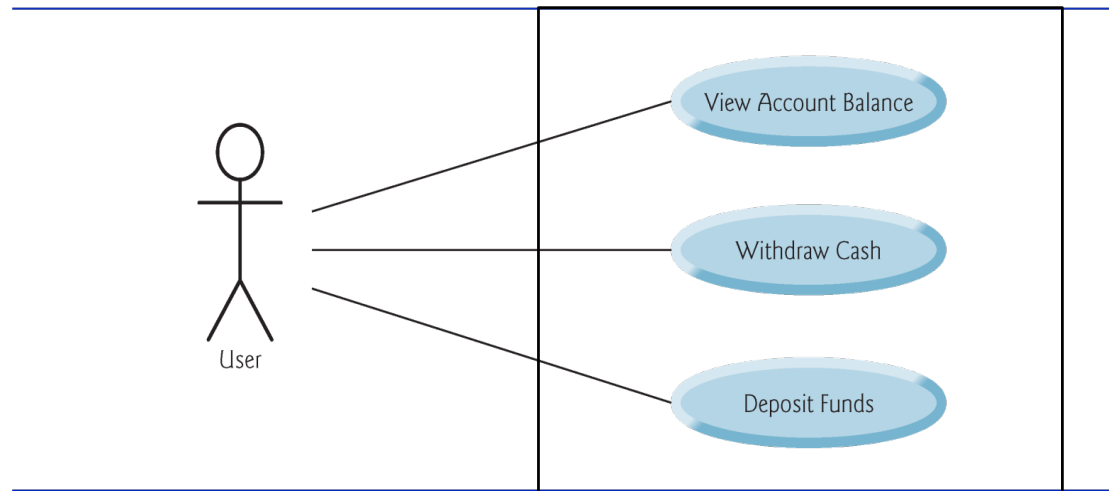


Fig. 12.4 | Use case diagram for the ATM system from the User's perspective.

... Examining the Requirements Document

- During the analysis stage, systems designers focus on understanding the requirements document to produce a high-level specification that describes what the system is supposed to do
- The output of the design stage—a **design specification**—should specify clearly **how** the system should be constructed to satisfy these requirements
- In the next several sections, we perform the steps of a simple object-oriented design (OOD) process on the ATM system to produce a design specification containing a collection of UML diagrams & supporting text

Identifying Classes in a Requirements Document

- Identify the classes that are needed to build the system by analyzing the *nouns & noun phrases* that appear in the requirements document
- We introduce UML class diagrams to model these classes
 - Important first step in defining the system's structure
- Review the requirements document & identify key nouns & noun phrases to help us identify classes that comprise the ATM system
 - We may decide that some of these nouns & noun phrases are actually attributes of other classes in the system
 - We may also conclude that some of the nouns do not correspond to parts of the system & thus should not be modeled at all
 - Additional classes may become apparent to us as we proceed through the design process

... Identifying Classes in a Requirements Document

- Class diagrams also show the relationships between the classes of the system
- Figure 12.7 shows how our classes ATM & Withdrawal relate to one another
- Notice that the rectangles representing classes in this diagram are not subdivided into compartments
- The UML allows the suppression of class attributes & operations in this manner to create more readable diagrams, when appropriate
- Such a diagram is said to be an **elided diagram**



Fig. 12.7 | Class diagram showing an association among classes.

... Identifying Classes in a Requirements Document

- The solid line that connects the two classes represents an **association**—a relationship between classes
- The numbers near each end of the line are **multiplicity** values, which indicate how many objects of each class participate in the association
 - At any given moment, one ATM object participates in an association with either zero or one Withdrawal objects—zero if the current user is not currently performing a transaction or has requested a different type of transaction, & one if the user has requested a withdrawal
- Figure 12.8 lists & explains the multiplicity types

Symbol	Meaning
0	None
1	One
m	An integer value
0..1	Zero or one
m, n	m or n
$m..n$	At least m , but not more than n
*	Any nonnegative integer (zero or more)
0..*	Zero or more (identical to *)
1..*	One or more

Fig. 12.8 | Multiplicity types.

Identifying Class Operations

- An operation is a service that objects of a class provide to clients (users) of the class
- We can derive many of the class operations by examining the key *verbs & verb phrases* in the requirements document
- The verb phrases in Fig. 12.16 help us determine the operations of each class

Class	Verbs and verb phrases
ATM	executes financial transactions
BalanceInquiry	<i>[none in the requirements document]</i>
Withdrawal	<i>[none in the requirements document]</i>
Deposit	<i>[none in the requirements document]</i>
BankDatabase	authenticates a user, retrieves an account balance, credits a deposit amount to an account, debits a withdrawal amount from an account
Account	retrieves an account balance, credits a deposit amount to an account, debits a withdrawal amount from an account
Screen	displays a message to the user
Keypad	receives numeric input from the user
CashDispenser	dispenses cash, indicates whether it contains enough cash to satisfy a withdrawal request
DepositSlot	receives a deposit envelope

Fig. 12.16 | Verbs and verb phrases for each class in the ATM system.

Indicating Collaboration Among Objects

- The UML provides several types of **interaction diagrams** that model the behavior of a system by modeling how objects interact
- The **communication diagram** emphasizes which objects participate in *collaborations*
- Like the communication diagram, the **sequence diagram** shows collaborations among objects, but it emphasizes when *messages* are sent between objects over time

... Indicating Collaboration Among Objects

➤ Communication diagram

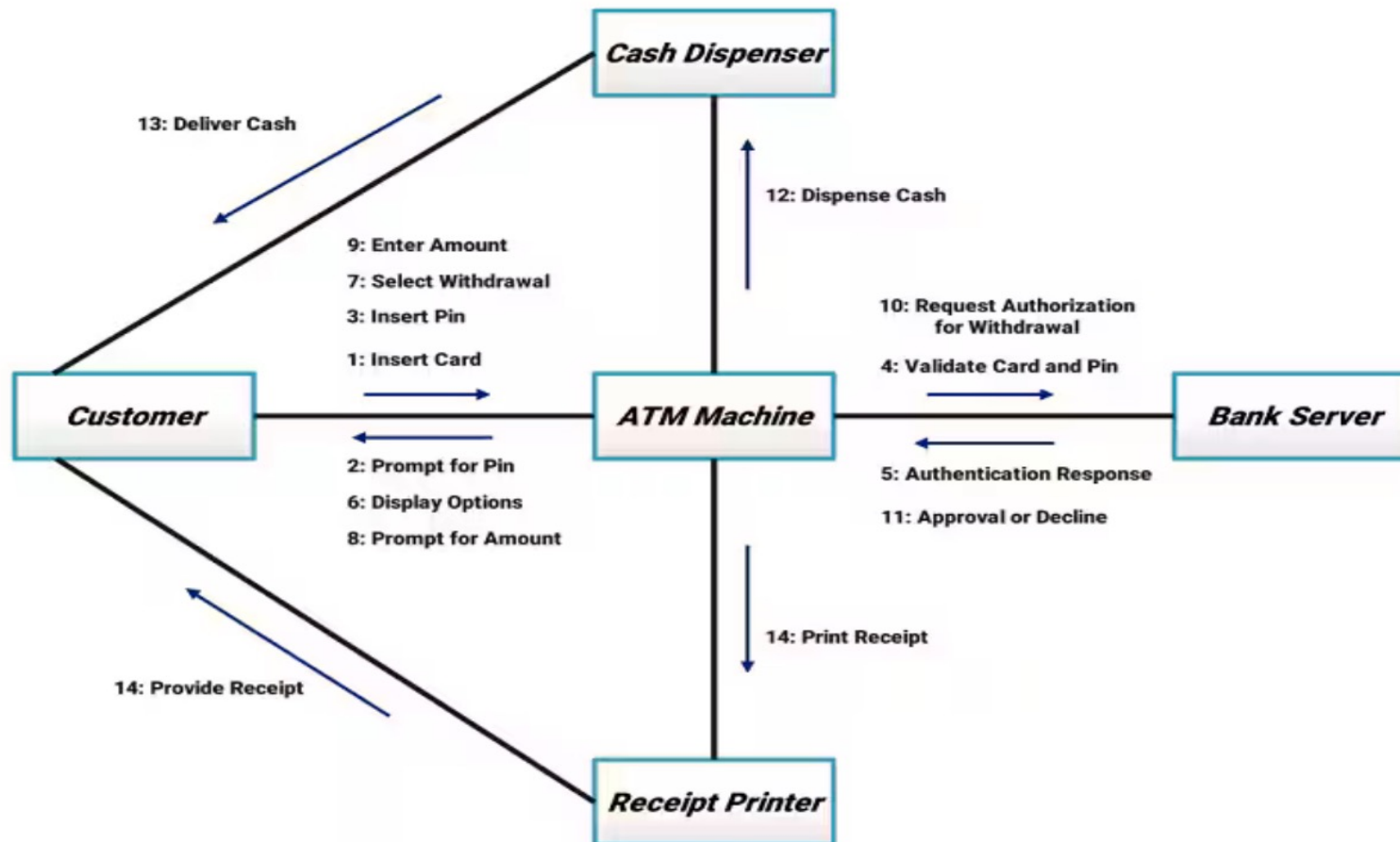
- Objects are modeled in the UML as rectangles containing names in the form `objectName: ClassName`
- When you have only one object of each type, you can disregard the object name & list only a colon followed by the class name
- Communicating objects are connected with solid lines, & messages are passed between objects along these lines in the direction shown by arrows
- The name of the message, which appears next to the arrow, is the name of an operation (i.e., a method in Java) belonging to the receiving object

... Indicating Collaboration Among Objects

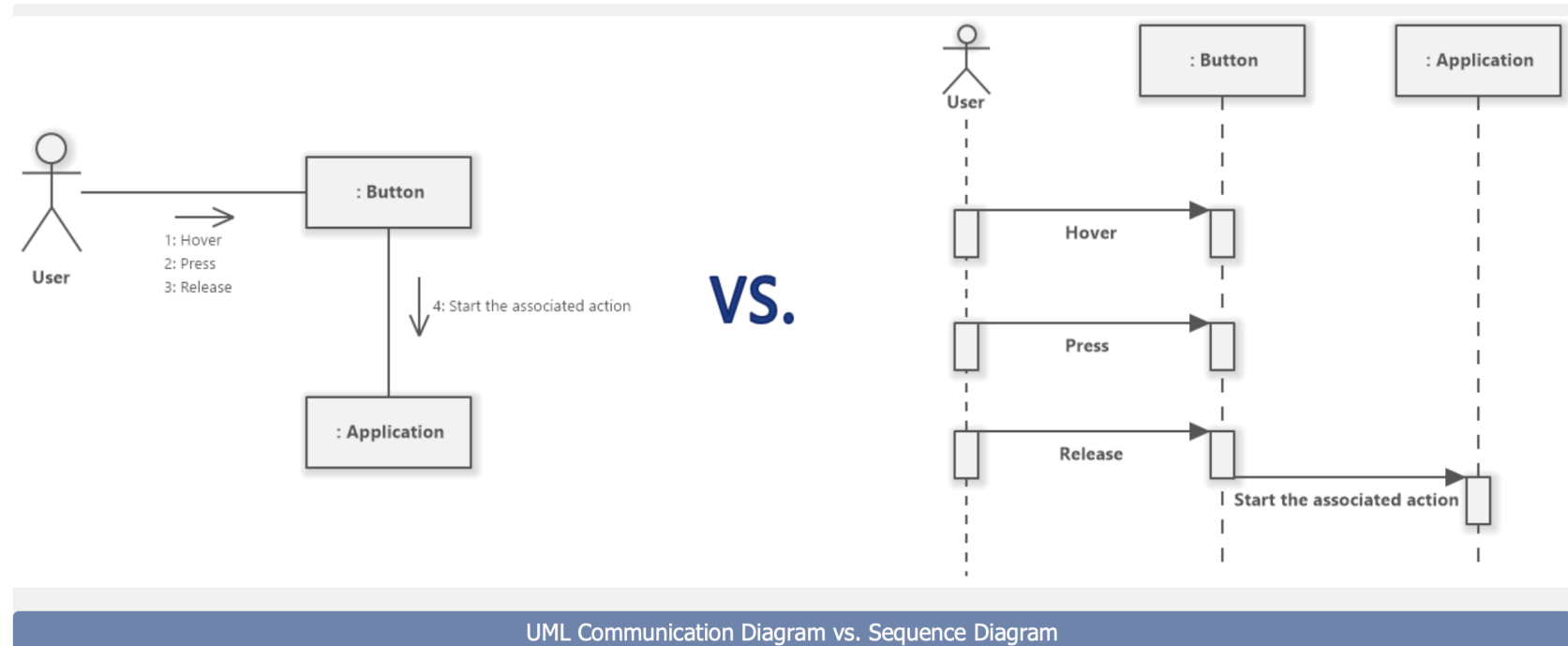
58

- Communication diagram
 - The solid filled arrow represents a message — or **synchronous call** —in the UML & a method call in Java. Flow of control is from the sending object to the receiving object.
 - **Asynchronous calls** are represented by a stick () arrowhead

Communication Diagram for ATM



Two common types of [UML diagrams](#) are **communication diagrams** and **sequence diagrams**. While they both provide a way to model interactions between objects or components in a system, they differ in their level of detail and focus.



Communication diagrams show the interactions between objects or components in terms of sequenced messages. They show the flow of messages between objects, along with the object that sends and receives the message. Communication diagrams also show the relationships between objects, including which objects are responsible for sending and receiving messages.

Sequence diagrams, on the other hand, focus on the time order of messages. They show the interactions between objects or components as a series of horizontal lines, with the object boxes listed at the top side-by-side. Messages are shown as horizontal arrows connecting the objects, and the order of the messages is shown by the vertical position on the diagram. Sequence diagrams also show the lifecycle of objects, including the creation and destruction of objects.

Key Points

- ✧ A software architecture is a description of how a software system is organized
- ✧ Architectural design decisions include decisions on the type of application, the distribution of the system, the architectural styles to be used
- ✧ Architectures may be documented from several different perspectives or views such as a conceptual view, a logical view, a process view, & a development view
- ✧ Architectural patterns are a means of reusing knowledge about generic system architectures. They describe the architecture, explain when it may be used & describe its advantages & disadvantages.

... Key Points

- ✧ Models of application systems architectures help us understand & compare applications, validate application system designs & assess large-scale components for reuse
- ✧ Transaction processing systems are interactive systems that allow information in a database to be remotely accessed & modified by a number of users
- ✧ Language processing systems are used to translate texts from one language into another & to carry out the instructions specified in the input language. They include a translator & an abstract machine that executes the generated language.

*** * * End *****